executed. It is typically a Razor Page, a web API controller action method, or a minimal API endpoint handler.

With this configuration, the `RoutingMiddleware` selects an endpoint to execute based on the request's URL, such as a Razor Page, as you saw in chapter 14. Metadata about the selected endpoint is available to all middleware that occurs after the routing middleware. This metadata includes details about any authorization requirements for the endpoint, and it's typically attached by decorating an action or Razor Page with an `[Authorize]` attribute.

The `AuthenticationMiddleware` deserializes the encrypted cookie (or bearer token for APIs) associated with the request to create a `ClaimsPrincipal`. This object is set as the `HttpContext.User` for the request, so all subsequent middleware can access this value. It contains all the `Claims` that were added to the cookie when the user authenticated.

NOTE Remember that the authentication middleware may be placed before the routing middleware when the authentication process is the same for all endpoints. Nevertheless, I prefer to place it as shown in figure 24.2, after the routing middleware, and always before the authorization middleware.

Now we come to the `AuthorizationMiddleware`. This middleware checks whether the selected endpoint has any authorization requirements, based on the metadata provided by the `RoutingMiddleware`. If the endpoint has authorization requirements, the `AuthorizationMiddleware` uses the `HttpContext.User` to determine whether the current request is authorized to execute the endpoint.